DISASTERRESOURCE+: A WEB-BASED RESOURCE OPTIMIZATION AND VISUALIZATION SYSTEM FOR DISASTER RESPONSE

A Capstone Project presented to the
Faculty of College of Computer Studies
Laguna State Polytechnic University
San Pablo City Campus

In partial Fulfillment of the Requirements for the degree
MASTER IN INFORMATION TECHNOLOGY

Emmanuel B. Manalo

September 2026

TABLE OF CONTENTS

Preliminaries **Page**

Title Page..... i

Table of Contents..... ii

List of Figures..... v

List of Tables..... vi

Definition of Terms..... vii

Acronyms..... viii

CHAPTER I..... 1

Research Problem..... 3

Research Objectives..... 6

Research Framework..... 7

Theoretical Framework..... 7

Conceptual Framework..... 9

Significance of the Research..... 14

CHAPTER II..... 17

Optimization Algorithms for Disaster Resource Allocation..... 17

Inventory Reorder-Point and Safety-Stock Theory..... 18

Offline-First Progressive Web App (PWA) Architecture..... 18

Related and Similar Disaster Management Systems..... 20

ISO/IEC 25010 Software Quality Evaluation..... 21

Synthesis..... 22

CHAPTER III..... 25

Research Design..... 25

Locale of the Studyheuristic..... 25

Applied Concepts and Techniques..... 26

Algorithm Analysis..... 27

Data Collection Methods..... 28

Optimization Model Generation..... 29

Data Collection and Cleaning..... 30

Parameter Preparation..... 30

Model Building..... 31

Model Evaluation..... 31

Model Deployment..... 31

System Development Methodology..... 32

Product Backlog..... 32

Sprint Planning..... 33

Sprint Execution..... 33

Sprint Review..... 33

Sprint Retrospective and Deployment..... 34

Software Tools Used..... 34

Hardware Requirements..... 35

System Architecture..... 36

Software Testing..... 38

Unit Testing Procedure..... 39

Integration Testing Procedure..... 39

Performance Testing Procedure..... 39

User Acceptance Testing Procedure..... 40

LIST OF FIGURES

Figure No.	Title	Page
1	Theoretical Framework	9
2	Conceptual Framework	11
3	System Architecture of DisasterResource+	38

LIST OF TABLES

Table No.	Title	Page
1	Identified Technology Gaps in Existing Disaster Resource Management Approaches	24
2	Dataset Attributes Description	29
3	Sample Trailing-Quarter Consumption and Suggested Reorder	32
4	Development Machine Minimum Specification	36

DEFINITION OF TERMS

The following terms were conceptually and operationally defined for the purpose of the research study.

Allocation	The assignment of a specific quantity of a specific resource type from a specific depot to a specific request, produced either by the optimization engine (as a suggestion) or confirmed by coordinator approval.
Borrowed-Item Lifecycle	The sequence of statuses a borrowed item passes through after issuance: issued, returned, for repair, condemned (written off), or surrendered
Borrowed Request	A resource request for equipment or items that are lent out and expected to be returned, requiring identification and contact details of the requester for accountability purposes
CDRRMO	City Disaster Risk Reduction and Management Office; the city-level implementing body under the LDRRMC structure established by RA 10121, and the primary stakeholder and pilot site (San Pablo City) for this study.
Consumable Request	A resource request for items that are used up during operations (e.g., fuel, medical supplies, food packs) and are not expected to be returned.

IndexedDB	A browser-based local storage system used by the offline-first Progressive Web App to queue field data entries when network connectivity is unavailable.
LDRRMC	Local Disaster Risk Reduction and Management Council; the general term for the council structure (regional, provincial, city/municipal, or barangay level) established under RA 10121.
Optimization Engine	The subsystem that applies linear programming (via PuLP) to compute a suggested budget allocation across resource types, based on trailing-quarter consumption, current stock, and the submitted procurement budget.
Progressive Web App (PWA)	A web application built to function offline-first, using local caching so that field data logging continues uninterrupted during telecommunication blackouts.
PSGC	Philippine Standard Geographic Code; the official geographic coding system maintained by the Philippine Statistics Authority, used in this study for standardized location data (region, province, city/municipality, barangay).

ACRONYMS

CDRRMO	City Disaster Risk Reduction and Management Office
LDRRMC	Local Disaster Risk Reduction and Management Council
LGU	Local Government Unit
ERT	Emergency Response Team
PWA	Progressive Web App
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission
SDG	Sustainable Development Goal
LP	Linear Programming
API	ApplicatiWeb-GISon Programming Interface
PSGC	Philippine Standard Geographic Code

CHAPTER I

INTRODUCTION AND ITS BACKGROUND

Disaster risk reduction and resource management have become critical priorities globally as climate change increases the frequency and severity of natural disasters. Internationally, modern emergency logistics frameworks increasingly rely on automated supply chain optimization and Geographic Information Systems (GIS) to accelerate relief distribution and minimize operational bottlenecks. In the Philippines, local government units (LGUs) serve as the primary responders during calamities, tasked with distributing life-saving supplies to affected populations under severe pressure. Despite advancements in digital technology, local response units continue to face systemic coordination challenges, manual tracking errors, and critical communication breakdowns when extreme weather events disrupt municipal power grids and cellular networks.

In San Pablo City, the Disaster Risk Reduction and Management Office (CDRRMO) plays a central role in managing evacuation centers, tracking supply inventories, and coordinating relief operations across local barangays. However, the current operational workflow relies heavily on manual logbooks, static spreadsheets, and decentralized communication channels. During active emergency responses, this manual setup creates severe operational friction, including allocation imbalances across evacuation centers, delayed distribution of emergency rations and medical supplies, and localized resource duplication. Furthermore, when local telecommunications infrastructure fails during severe storms, tracking capability drops significantly, creating operational blind spots that

severely hinder emergency decision-makers from maintaining real-time situational awareness.

This research directly aligns with the United Nations Sustainable Development Goals (SDGs), specifically SDG 11: Sustainable Cities and Communities (Target 11.5) and SDG 13: Climate Action. By strengthening local disaster preparedness and optimizing crisis logistics, the proposed solution helps safeguard vulnerable communities against disaster-induced disruptions. Furthermore, by improving supply chain efficiency and reducing material wastage during emergency relief operations, the study contributes to SDG 12: Responsible Consumption and Production. Integrating these global targets into local disaster governance ensures that emergency response strategies are both resilient and sustainable.

To bridge the gap between static manual operations and dynamic disaster response, there is an urgent necessity to develop DisasterResource+. The technical gap lies in the absence of a unified, offline-resilient web application capable of combining spatial intelligence with dynamic decision support. Traditional desktop mapping and standalone spreadsheet tools lack the ability to compute real-time supply allocations or preserve data integrity during network blackouts. DisasterResource+ addresses these limitations by introducing a web-based platform built on a Progressive Web App (PWA) architecture with IndexedDB caching, enabling continuous field data entry and automated synchronization once network connectivity is restored.

To solve this problem, DisasterResource+ incorporates a Linear Programming allocation model built with PuLP. Given each resource type's historical quarterly consumption, current depot stock, and a Logistics-entered procurement budget or order-capacity constraint, the model computes an optimized replenishment allocation that minimizes stockout risk for the next quarter. By automating this calculation, the system removes the manual counting and estimation errors that currently drive CDRRMO San Pablo City's reorder decisions. Paired with a real-time analytics dashboard visualizing stock levels, utilization, and the resulting reorder recommendation, the software equips CDRRMO Logistics personnel with actionable, data-driven inventory decision support.

Research Problem

The general problem of this study is the inefficient, delayed, and uncoordinated emergency relief resource allocation and tracking within the San Pablo City Disaster Risk Reduction and Management Office (CDRRMO San Pablo City) during calamity response operations. Specifically, the study seeks to answer the following questions:

- How can an automated decision-support model be designed using linear programming to optimize quarterly resource replenishment allocation, given historical consumption, current stock levels, and a procurement budget or order-capacity constraint?

- How can an offline-first Progressive Web App (PWA) architecture be implemented using local caching (IndexedDB) to maintain continuous inventory data logging and tracking during telecommunication blackouts?
- How can a real-time analytics dashboard be developed to give Logistics and Research and Planning personnel clear visibility into historical versus current depot stock levels, resource utilization, and the system's quarterly reorder recommendation?
- How can the system manage the full resource-request and inventory lifecycle incident — distinguishing consumable from borrowed-item requests with lifecycle tracking, automatically reserving and deducting depot stock upon request approval and fulfillment, and generating a historical-consumption-informed quarterly reorder advisory with an exportable burn-rate report — to replace CDRRMO San Pablo City's reliance on manual logbooks and spreadsheets?
- How acceptable is the DisasterResource+ system to CDRRMO San Pablo City personnel based on the International Organization for Standardization/International Electrotechnical Commission (ISO/IEC) 25010 software quality standards in terms of functional suitability, reliability, and usability?

The root cause of this problem stems from the reliance on manual logbooks, static spreadsheets, and decentralized verbal communication channels during active disaster operations. As a result, local responders experience

severe operational bottlenecks, including relief supply delivery delays, resource duplication, and unbalanced inventory distribution across evacuation centers. When severe weather causes electrical power failures and network downtime, tracking capabilities collapse entirely, creating dangerous operational blind spots that slow down critical decision-making and delay assistance to affected families.

This problem is strongly evidenced by global and local disaster response literature. According to studies on humanitarian supply chain management, traditional relief distribution networks frequently suffer from the “bullwhip effect” and delayed operational feedback, leading to up to a 40% reduction in response efficiency during peak emergency windows. Furthermore, field evaluations of local disaster risk reduction management units in the Philippines show that manual data processing during major typhoons leads to significant discrepancies between reported relief requests and actual ground delivery, directly compromising community recovery efforts. This was independently confirmed during stakeholder consultation with CDRRMO San Pablo City’s logistics section, which identified manual human counting error as the most persistent and recurring source of inventory inaccuracy in current operations, and named inventory automation as the single highest-priority improvement the office needs, ahead of any other proposed feature.

To solve this problem, the researchers propose DisasterResource+, a web-based resource optimization and visualization system that utilizes linear programming algorithms, an inventory visualization dashboard, and an offline-first Progressive Web App architecture. This strategy automates quarterly

procurement budget allocation and enables continuous depot data logging to ensure accurate stock tracking even during network outages..

Research Objectives

The general objective of this study is to design, develop, and evaluate DisasterResource+, a web-based resource optimization and visualization system for the San Pablo City Disaster Risk Reduction and Management Office (CDRRMO San Pablo City) to improve quarterly resource replenishment decisions, inventory tracking resiliency, and Logistics decision-making.

Specific Objectives:

Specifically, the study aims to:

1. Design and develop an automated decision-support engine using linear programming to optimize quarterly resource replenishment allocation, given historical consumption, current stock levels, and a procurement budget or order-capacity constraint.
2. Implement an offline-first Progressive Web App (PWA) architecture with local data caching (IndexedDB) to enable continuous inventory data logging and tracking during telecommunication blackouts.
3. Build a real-time analytics dashboard to give Logistics and Research and Planning personnel clear visibility into historical versus current depot stock levels, resource utilization, and the system's quarterly reorder recommendation.
4. Design and implement a resource-request and inventory management module that distinguishes consumable from borrowed-item

requests with lifecycle status tracking, automatically reserves and deducts depot stock upon request approval and fulfillment, and generates a historical-consumption-informed quarterly reorder advisory with an exportable Inventory Burn-Rate & Reorder Report (PDF/CSV).

5. Evaluate the software quality and acceptability of DisasterResource+ among CDRRMO San Pablo City personnel based on the ISO/IEC 25010 standard in terms of functional suitability, reliability, and usability.

Research Framework

This study rests on two theories. Each one supports a different part of DisasterResource+. Optimization theory supports the replenishment engine. Offline-first computing theory supports the inventory data layer."

Theoretical Framework

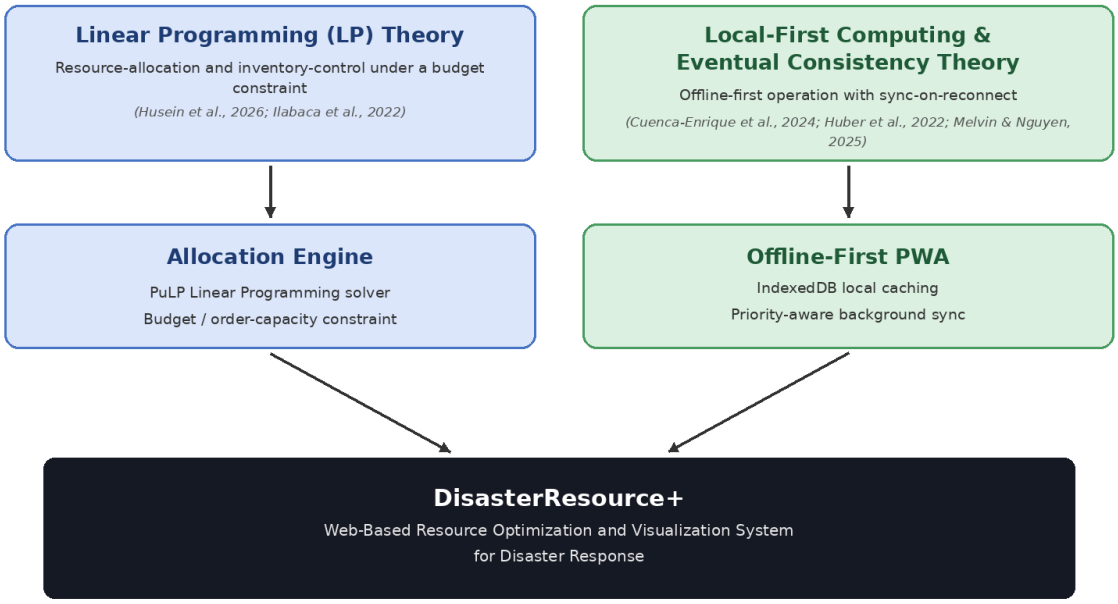
This study rests on two theories. Each one supports a different part of DisasterResource+. Optimization theory supports the replenishment engine. Offline-first computing theory supports the inventory data layer.

The replenishment engine is based on Linear Programming (LP) theory, the same math used to solve resource-allocation and inventory-control problems under a budget constraint (Husein et al., 2026; Ilabaca et al., 2022). LP theory gives a clear way to decide how to spend a limited procurement budget across resource types. The goal is to minimize stockout risk for the next quarter while

staying within the budget or order-capacity limit and using each resource type's trailing-quarter consumption to guide how much of it to reorder.

The offline-first PWA design is based on the theory of eventual consistency and local-first computing. This theory says a system should keep working from its own local data even when it loses connection to a central server. Once the connection comes back, the system can sync everything again instead of needing constant connectivity just to function (Cuenca-Enrique et al., 2024; Huber et al., 2022). This idea shapes how DisasterResource+ uses IndexedDB. The system stores inventory data locally during telecommunication blackouts and syncs it once the network returns. When bandwidth is limited, safety-critical data is given priority over routine content (Melvin & Nguyen, 2025).

Figure 1
Theoretical Framework of DisasterResource+



Conceptual Framework

Building on the theoretical foundations above, DisasterResource+'s conceptual framework follows an Input–Process–Output (IPO) model, illustrated in Figure 2.

Input. The system takes as input: (1) each resource type's historical quarterly consumption, drawn from prior QuarterlyStockSnapshot records; (2) current depot-level stock (resource type, quantity available); and (3) a Logistics-entered procurement budget or order-capacity constraint for the upcoming quarter.

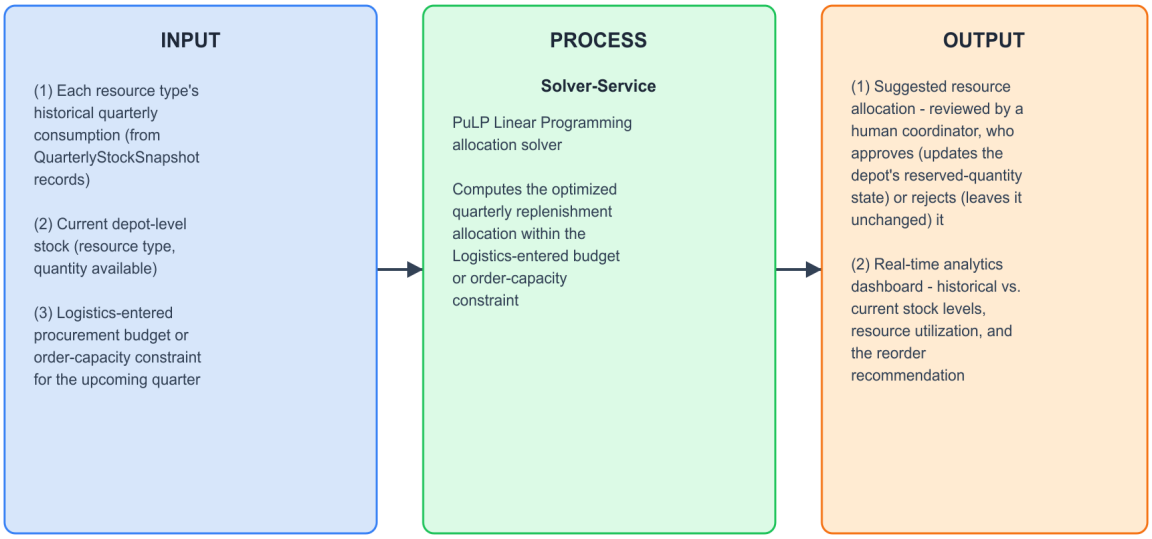
Process. The core process is the quarterly replenishment cycle: at the start of each quarter, the system sends each resource type's consumption history, current stock, and the budget/order-capacity constraint to the

solver-service, which applies a PuLP-based linear programming model to compute an optimized replenishment allocation that minimizes stockout risk across resource types while staying within the budget.

Output. The process yields: (1) a suggested (not auto-applied) replenishment allocation, presented to a human coordinator for review and approval, with an approval updating the depot's reserved-quantity state and a rejection leaving it unchanged; and (2) a real-time analytics dashboard showing historical versus current stock levels, resource utilization, and the reorder recommendation.

This conceptual framework is represented as a diagram in Figure 2. It begins with the quarterly input, meaning each resource type's consumption history and current depot stock. That data then moves into the Solver-Service process, where the PuLP linear programming model computes the optimized replenishment allocation within the budget constraint. From there, the process produces two outputs: a suggested allocation waiting for the coordinator's approval, and an analytics dashboard showing the current inventory situation. The coordinator's decision on that suggested allocation is the final step of the output: approval updates the depot and resource state, while rejection leaves them unchanged.

Figure 2
Conceptual Framework of DisasterResource+ (IPO MODEL)



Scope and Limitations of the Research

Scope. This study covers the design, development, and evaluation of DisasterResource+, a web-based resource optimization and visualization system built for the San Pablo City Disaster Risk Reduction and Management Office (CDRRMO San Pablo City). The system covers four main areas. The first is an automated linear-programming engine that allocates a Logistics-entered procurement budget or order-capacity constraint across resource types each quarter, based on historical consumption and current depot stock. The second is an offline-first Progressive Web App with IndexedDB-based local caching, which keeps inventory data logging running during telecommunication blackouts. The third is a real-time analytics dashboard that gives Logistics and Research and Planning personnel visibility into historical versus current stock levels, resource utilization, and the reorder recommendation. A fourth area covers request

accountability: the system distinguishes consumable from borrowed-item requests, each with its own control-number scheme, and tracks borrowed equipment through a defined lifecycle (issued, returned, for repair, condemned, or surrendered), directly addressing the accountability gap identified during consultation with CDRRMOO San Pablo City. Respondents for the acceptability evaluation are limited to CDRRMO San Pablo City personnel directly involved in its emergency operations. They will be evaluated against the ISO/IEC 25010 standard, specifically on functional suitability, reliability, and usability. The platform also includes role-based access control, which lets system administrators from each participating agency manage user access and permissions instead of relying on shared or informal logins. These roles reflect CDRRMO San Pablo City's own functional divisions confirmed during stakeholder consultation — Incident Management, ERT (Emergency Response Team), Logistics, Operations, Research and Planning, and Admin & Training — rather than the ICS command structure used only during live disaster response, which this study does not model. Resource ownership in San Pablo City is not centralized in CDRRMO alone — the City Social Welfare and Development Office (CSWDO), City Health Office (CHO), and General Supply Office (GSO) each maintain separate stock that CDRRMO cannot view except through a formal request. Based on this stakeholder consultation, the system's inventory and optimization scope is intentionally limited to CDRRMO-held resources; cross-department stock visibility, while recognized as the more operationally useful long-term state, is deferred to future work. Barangay-level requests for

resources are also outside this study's scope: under existing CDRRMO policy, each barangay is required to allocate five percent of its own annual budget to a dedicated Disaster Risk Reduction and Management (DRRM) fund and is not expected to request supplementary resources directly from CDRRMO, so the system's request workflow models inter-division and inter-office coordination within CDRRMO's own operations rather than barangay-initiated requests.

Limitations. This study does not claim to build a fully autonomous procurement system. Every solver-suggested replenishment allocation still needs human coordinator approval before it counts as an approved reorder, and the system does not track real-time depot inventory changes that happen outside the application, such as a manual withdrawal that never gets logged. The replenishment model's reliability depends on the quality of the procurement budget or order-capacity figure Logistics enters and on having enough trailing-quarter consumption history for a resource type; a newly added resource type with no prior QuarterlyStockSnapshot records falls back to a zero-suggestion cold start, the same limitation already documented for the underlying reorder calculation. The system also does not model the Incident Command System (ICS) used during actual disaster response operations, since stakeholder consultation confirmed ICS is a separate, situational structure distinct from CDRRMO's standing organizational structure, which governs day-to-day office operations rather than live incident response.

Significance of the Research

This study will benefit the following stakeholders, presented from a macro to micro perspective:

Local Government Units (LGUs) and the CDRRMO San Pablo. This study supports the CDRRMO's core mandate directly. It replaces manual logbooks and static spreadsheets with an automated, quarterly resource-replenishment and tracking system. This should reduce stockouts, resource duplication, and manual counting errors in day-to-day inventory management. It also gives the office a tool that keeps running through the very network disruptions that usually come with severe disasters.

System Administrators (IT Support). Personnel responsible for keeping the platform running across participating agencies get role-based access control to manage who can view or edit what, along with a system built toward the ISO/IEC 25010 standard for stability and compliance with official DRRM data-handling guidelines.

Logistics Division. As the office directly responsible for day-to-day resource tracking, requests, and approvals, Logistics staff get an automated inventory system that replaces manual Excel sheets and physical logbooks, removing the human counting errors identified during stakeholder consultation as the office's single most persistent operational problem. This directly answers Logistics' own stated top priority for automation.

Research and Planning Division. These stakeholders get a real-time analytics dashboard showing historical versus current stock levels and the

system's quarterly reorder recommendation. This replaces the annual manual request for updated figures from Logistics with a constantly available, data-driven basis for procurement planning.

Residents and Evacuees of San Pablo City. As the people who ultimately benefit from well-stocked, well-managed depots, affected residents and evacuees should experience fewer shortages of emergency rations, medical supplies, and other relief resources when the office responds to a calamity.

The Academic and Research Community. This study documents a real case of applying linear-programming-based inventory optimization, offline-first PWA architecture, and analytics-dashboard visualization together within one municipal-level disaster-preparedness office. This adds to the still-limited body of literature (see Chapter II) that studies these components as one system instead of studying them separately.

Broader Societal and Sustainable Development Goals. As explained earlier in this chapter, this study supports SDG 11 (Sustainable Cities and Communities, Target 11.5), SDG 13 (Climate Action), and SDG 12 (Responsible Consumption and Production). It does this by strengthening local disaster preparedness, reducing material wastage in emergency relief operations, and helping build more resilient local disaster governance.

Taken together, the additions above shift the system's core contribution: rather than relying solely on a one-time allocation optimization for a single incident, DisasterResource+ now closes the loop between a suggested allocation and the depot's actual recorded stock, and gives Logistics staff a historical,

exportable basis for adjusting the next quarter's procurement request, directly answering the stakeholder's named top priority of inventory tracking rather than treating it as secondary to the optimization engine.

CHAPTER II

REVIEW OF LITERATURE

This chapter reviews foreign and local literature and studies that relate to the design and development of DisasterResource+. The review is grouped into four main topics, and each one matches a core part of the system. These are (1) optimization algorithms for inventory replenishment and stock allocation, (2) offline-first Progressive Web App (PWA) architecture, (3) related and similar disaster and local-government logistics management systems, and (4) ISO/IEC 25010 software quality evaluation. Each topic looks at research, algorithms, and software applications that connect closely to this study, and ends with a synthesis that points out the gap DisasterResource+ addresses.

Optimization Algorithms for Disaster Resource Allocation

Recent research on disaster logistics often uses Linear Programming (LP) to model relief distribution as a transportation problem with set limits. Husein et al. (2026) used an LP-based transportation model to move aid from several warehouses to affected shelters, working within real field conditions. Their approach cut costs by 58.06 percent while still meeting every supply and demand requirement. This shows that even a fairly simple LP setup can create large efficiency gains compared to distribution decided manually and without a system. DisasterResource+ aims for that same kind of gain through its PuLP-based replenishment solver, which allocates a quarterly procurement budget across resource types for CDRRMO San Pablo City’s depots.

Inventory Reorder-Point and Safety-Stock Theory

Reorder-point and safety-stock models are a foundational technique in inventory control theory, used to determine when and how much of a resource to replenish based on historical demand or consumption and a desired service level (Silver, Pyke, & Peterson, 1998). DisasterResource+ adapts this concept at a reduced scope appropriate to a single-office deployment: rather than a full statistical safety-stock calculation, it computes a suggested reorder quantity from trailing-quarter consumption averages, matching the granularity of CDRRMO San Pablo City's own existing quarterly review cycle rather than introducing an unfamiliar model. This directly formalizes a practice stakeholder consultation confirmed is already performed manually: stock levels from one quarter are used to adjust the request for the next quarter, which helps reduce the risk of surplus or shortage.

Offline-First Progressive Web App (PWA) Architecture

A major technical problem for disaster response software is that the network infrastructure it relies on is often the first thing the disaster knocks out. Cuenca-Enrique et al. (2024) studied Progressive Web Apps as a solution for rural communities in developing countries with unstable connectivity. They found that PWAs combine installability, offline caching, and tolerance for low bandwidth, which makes them a more practical fit than native mobile apps in places where a steady connection cannot be assumed. This finding applies directly to San Pablo City's barangay-level field operations during severe weather. Huber et al. (2022)

support this point too. They compared the energy and resource use of PWAs against native and hybrid alternatives across real-world applications and found that PWAs use a bit more energy than fully native apps, but remain a good option since one offline-capable codebase can serve desktop and mobile users without needing separate builds for each platform.

Melvin and Nguyen (2025) proposed AWARE, a priority-aware caching method called PriorityFresh, built specifically for offline emergency warning systems. Their argument is that not all cached data should be treated the same way when connectivity drops. Low-stock alerts and pending quarterly budget submissions should be kept fresh and given priority over routine content when storage or sync bandwidth runs short. This idea points to a real gap in DisasterResource+'s current IndexedDB caching layer, which queues field data for syncing but does not yet sort that data by priority (see Synthesis below). This need is not only a literature-supported design choice but a directly stated operational requirement: CDRRMO San Pablo City's logistics section confirmed during stakeholder consultation that any automated system must continue operating through both internet and power outages, and that the office currently relies on a deliberate sequential backup chain (digital system, then Excel spreadsheet, then paper log) specifically to remain functional when technology fails. DisasterResource+'s offline-first PWA architecture is designed to preserve this same fail-safe principle rather than replace it outright.

Related and Similar Disaster Management Systems

Several recent Philippine LGU-level ICT projects tackle parts of the problem DisasterResource+ is trying to solve, and looking at them helps show where this study's contribution fits in. Macose (2024) documented an ICT-powered Local Government Unit disaster solution built around community-based reporting and coordination. The study found that digitizing an LGU's disaster workflow clearly improved community resilience and response coordination, but the system still depended on manually reviewed reports instead of an automated inventory-replenishment-optimization layer. Albano (2025) built a similar kind of system, a dynamic, open-source web-based information system for a Philippine municipal LGU in Cabagan, Isabela, meant to improve transparency and access to municipal services and data. This shows that open-source web technology is a practical, low-cost base for LGU-level digital systems, which supports DisasterResource+'s own choice to build on an open-source Laravel and Inertia stack instead of a proprietary platform.

Locally, the Philippine Council for Health Research and Development's E-LIGTAS platform (PCHRD, n.d.) shows the same pattern in a disaster-specific Philippine setting. Its web component tracks dispatch and resource allocation status, but the actual allocation decisions are still made manually by response coordinators instead of being computed. Taken together, this related-systems literature points to one consistent gap. Whether it is a general municipal information system (Albano, 2025), a community-based disaster coordination tool (Macose, 2024), or a national disaster-response platform like E-LIGTAS

(PCHRD, n.d.), existing Philippine LGU-level digital systems are strong at sharing information, improving transparency, or tracking activity. None of them pair that strength with an automated inventory-replenishment-optimization engine. That is the specific gap DisasterResource+ is meant to fill for CDRRMO San Pablo City.

ISO/IEC 25010 Software Quality Evaluation

DisasterResource+ is meant for real field use by CDRRMO personnel, not just as a proof of concept. Because of that, its acceptability needs to be checked against a recognized software quality standard instead of relying on informal feedback alone. ISO/IEC 25010:2023 (International Organization for Standardization, 2023) sets out a product quality model built around characteristics such as functional suitability, performance efficiency, compatibility, reliability, security, maintainability, portability, and interaction capability, which was called usability in the 2011 edition. This gives DisasterResource+'s own evaluation tool a standard vocabulary to build on. Rojas Enriquez et al. (2025) carried out a bibliometric and thematic analysis of more than 1,600 Scopus-indexed articles that apply ISO/IEC 25010, and found that research using the standard has grown steadily, now covering public administration, finance, education, and healthcare. They also found that studies usually narrow the standard's broad characteristic list down to just the subset that fits the system being evaluated, rather than applying all of it. This is why DisasterResource+'s own evaluation focuses only on functional suitability, reliability, and usability

instead of the full eight-characteristic set, since the study cares about field-usability acceptance and not, for example, portability across platforms.

Real-world applications of ISO/IEC 25010 show how the standard works as a practical acceptance-evaluation tool. Pratama and Mutiara (2021) applied ISO 25010:2011 to evaluate the Halodoc telehealth application. They used structured user assessments across several quality characteristics to produce a measured acceptability rating instead of a purely descriptive review. Closer to DisasterResource+'s own setting, Osio et al. (2026) evaluated a web-based customer satisfaction survey system built for a Philippine local government unit using ISO/IEC 25010, with LGU staff and end-users as respondents. This closely matches how DisasterResource+ plans to evaluate its own CDRRMO personnel. Across these studies, the method stays consistent: pick a relevant subset of ISO/IEC 25010 characteristics, turn each one into survey items, and collect ratings from the people who will actually use the system. That is the same model DisasterResource+'s own quality evaluation, Research Objective 4, is built to follow.

Synthesis

The literature reviewed here shows that each individual technical piece of DisasterResource+ is already well studied and proven on its own. This includes LP-based resource-allocation and replenishment algorithms, offline-first PWA architecture, and ISO/IEC 25010-based quality evaluation. Linear programming is shown again and again to reduce cost and unmet demand when applied to resource allocation and distribution problems (Husein et al., 2026). Offline-first

PWA design, supported by IndexedDB and priority-aware caching, is proven to be a practical fit for field operations with limited connectivity (Cuenca-Enrique et al., 2024; Huber et al., 2022; Melvin & Nguyen, 2025). And ISO/IEC 25010 offers a proven method for checking field acceptability with real government end-users (Osio et al., 2026; Pratama & Mutiara, 2021; Rojas Enriquez et al., 2025).

What the literature does not show, though, is a single system that brings all three of these pieces together for a Philippine LGU-level disaster-logistics office. The related-systems review (Albano, 2025; Macose, 2024; PCHRD, n.d.) shows that existing Philippine LGU-level digital platforms are strong at sharing information, improving transparency, or tracking activity, but none of them go as far as pairing that strength with an automated LP-based replenishment engine that also works offline-first and gets evaluated against a formal software quality standard by its own field users. That is the specific gap DisasterResource+ is built to close. It is an offline-resilient, PWA-based web application that brings together automated, budget-constrained inventory replenishment optimization and ISO/IEC 25010-based acceptability evaluation in one system built specifically for a municipal-level CDRRMO's logistics operations. Table 1 below summarizes this gap.

Table 1
Identified Technology Gaps in Existing Disaster Resource Management Approaches

Aspect	Current State	Gap/Limitation	Impact of Gap	References
Resource allocation	LP-based optimization exists but is studied as a standalone algorithmic contribution, rarely embedded in a fielded LGU-level system	Optimization research and operational disaster-response software remain largely separate tracks	LGU responders continue to allocate resources manually even where optimized methods are proven effective	Husein et al., 2026; Macose, 2024
Offline resilience	PWA/IndexedDB offline patterns are validated for general and rural use cases	Priority-aware retention (e.g., AWARE's PriorityFresh) is rarely implemented in disaster-specific offline caching	Field data can be lost or synced out of priority order during prolonged network outages	Cuenca-Enrique et al., 2024; Melvin & Nguyen, 2025
Quality evaluation	ISO/IEC 25010 is an established method for evaluating software with real end-users, including in Philippine LGU contexts	Rarely applied to disaster-response-specific systems evaluated with CDRRMO respondents	Disaster software often reaches deployment without a standardized acceptability evaluation from its actual field users	Osio et al., 2026; Rojas Enriquez et al., 2025

CHAPTER III

RESEARCH METHODOLOGY

This chapter outlines the research methodology employed in this study, detailing the approaches and procedures used to design, develop, and evaluate DisasterResource+.

Research Design

This study adopts a hybrid research design combining Developmental and Descriptive-Evaluative approaches. The developmental component focuses on designing and building DisasterResource+, a web-based resource optimization and inventory visualization system composed of an automated, linear-programming-based inventory replenishment engine, an offline-first Progressive Web App (PWA), and an inventory visualization dashboard. The descriptive-evaluative component assesses the finished system's acceptability among CDRRMO San Pablo City personnel using the ISO/IEC 25010 standard, specifically on functional suitability, reliability, and usability. This design ensures a structured process for building the system while empirically assessing whether it meets the study's objectives.

Locale of the Studyheuristic

This research will be conducted for the City Disaster Risk Reduction and Management Office (CDRRMO) of San Pablo City, Laguna. CDRRMO San Pablo City is the primary stakeholder and pilot site for this study, having its own depot inventory and quarterly procurement process managed by its Logistics section.

The study's evaluation respondents are limited to CDRRMO San Pablo City personnel directly involved in its logistics and inventory operations, as confirmed through stakeholder consultation conducted at the CDRRMO San Pablo City office.

Applied Concepts and Techniques

The core concepts underpinning this research involve Operations Research, specifically Linear Programming (LP) applied to budget-constrained inventory replenishment, combined with offline-first web architecture and analytics-dashboard decision support.

Operations Research → Optimization → Linear Programming → Budget-Constrained Replenishment Problem. The replenishment engine is formulated as a constrained optimization problem: deciding an order quantity per resource type so that stockout risk for the next quarter is minimized while respecting the Logistics-entered procurement budget or order-capacity limit. This is implemented using PuLP, an open-source LP modeling library for Python, which formulates the allocation as decision variables, an objective function, and a set of constraints, then solves it using the CBC (Coin-or Branch and Cut) solver.

Distributed Systems → Offline-First Computing → Local-First Storage. The inventory data-logging layer is grounded in offline-first/local-first computing theory, implemented using the browser's IndexedDB storage so that depot and request data entries queue locally during telecommunication blackouts and synchronize automatically once connectivity is restored.

Algorithm Analysis

Since DisasterResource+'s core allocation problem is a constrained linear optimization problem rather than a classification or regression task, its solver is analyzed using operations-research performance criteria rather than machine-learning classification metrics (e.g., accuracy, precision, recall are not applicable here since there is no labeled ground-truth classification being predicted).

The allocation solver's quality will be measured using the following criteria:

Projected Stockout Ratio (PSR). The proportion of resource types projected to fall below their safety-stock target next quarter even after the optimized replenishment allocation is applied, expressed as:

PSR = (Resource Types Below Target After Allocation) / Total Resource Types

A lower PSR indicates a more effective replenishment allocation.

Allocation Feasibility. Whether a proposed solution satisfies all constraints (procurement budget or order-capacity limit, non-negative order quantities) is verified for every solver run; an infeasible result is treated as a solver failure requiring investigation.

Solver Runtime. Since PuLP's underlying CBC solver uses the Simplex method and Branch-and-Bound for the LP/MILP formulation, the solve time is measured directly (in milliseconds) across increasing problem sizes (number of

resource types times number of depots) to characterize the solver's practical time complexity, denoted as O(n) relative to the number of decision variables n.

The criteria the algorithm must satisfy — input (valid depot/resource/budget data), output (a feasible or explicitly infeasible allocation result), definiteness (the same input always produces the same allocation), finiteness (the solver terminates within a bounded time), and effectiveness (the result reduces stockout risk relative to no optimization) — will all be verified during testing.

Data Collection Methods

Data for this study comes from the system's own operational records: depot-level resource stock (resource type, quantity available), resource requests and their approval/fulfillment status, and quarterly consumption snapshots, all logged by CDRRMO Logistics staff, plus the Logistics-entered procurement budget or order-capacity figure for each upcoming quarter. These are structured, quantitative datasets suitable for the LP solver's input requirements. Table 2 summarizes the core dataset attributes used by the replenishment engine.

Table 2
Dataset Attributes Description

Attribute	Description	Sample Values
depot_id	Unique identifier for a resource depot	D-002
resource_type_id	Identifier for the type of resource tracked at a depot	RT-004 (e.g., food packs)
year	Calendar year of the quarterly consumption snapshot	2026
quarter	Quarter number (1-4) the consumption snapshot covers	2
quantity_available_at_close	Stock on hand for a resource type at a depot when the quarter closed	50, 120
quantity_consumed	Amount of a resource type consumed during the quarter	80, 120
budget_amount	Logistics-entered procurement budget for the depot for the upcoming quarter	PHP 50,000

This data will be gathered as Logistics staff use DisasterResource+ during actual and simulated quarterly replenishment cycles over the course of system testing and evaluation.

Optimization Model Generation

This section outlines the framework used to generate the allocation engine's suggested quarterly replenishment allocations, following a process

adapted from the standard machine-learning/data-science pipeline (data collection, preparation, model building, evaluation, and deployment), applied here to an operations-research optimization model rather than a predictive model.

Data Collection and Cleaning

Each resource type's trailing-quarter consumption history (QuarterlyStockSnapshot: quantity consumed and quantity available at close, per depot, per quarter) and current on-hand stock (app_resources.quantity_available) are pulled directly from the database. Before the LP runs, this data is validated to ensure every resource type has at least one prior quarter of snapshot history and that quantities are non-negative; resource types with no consumption history are flagged for manual review rather than given a computed reorder suggestion.

Parameter Preparation

Before the LP model can run, several derived parameters must be computed per resource type (and per depot, where stock is tracked at that granularity): a trailing-quarters average consumption, a safety-stock target derived from that average and a configurable service-level factor, and a unit cost used to convert an order quantity into its share of the procurement budget. These parameters feed directly into the LP objective and its budget constraint.

Model Building

The replenishment problem is formulated in PuLP as a linear program: decision variables represent the order quantity for each resource type (per depot, where applicable) for the coming quarter; the objective function minimizes total projected stockout risk, measured as the shortfall between projected next-quarter stock (current stock plus the ordered quantity) and each resource type's safety-stock target; constraints enforce that total order cost (order quantity times unit cost, summed across resource types and depots) does not exceed the admin-entered procurement budget, and that order quantities are non-negative.

Model Evaluation

The solver's suggested replenishment plan is evaluated by comparing the resulting Projected Stockout Ratio and solver runtime against the current deterministic baseline (a flat 1.5x safety-factor formula applied uniformly to trailing-quarter consumption) across a set of representative test scenarios built from CDRRMO San Pablo City's historical consumption and stock records.

Model Deployment

The finalized allocation logic is deployed as part of the solver-service, a FastAPI microservice invoked by a queued job whenever a Logistics coordinator submits or updates the next quarter's procurement budget. The solver-service exposes this logic through a REST endpoint, keeping the optimization engine decoupled from the main web application so it can be updated or re-tuned independently. Inventory reservation and deduction now closes the loop between

a suggested allocation and the depot's recorded stock: approving a resource request reserves the corresponding quantity, and confirming its fulfillment deducts it from stock on hand, replacing what was previously an unenforced, manually-tracked step. Table 3 illustrates a sample of this calculation for two resource types.

Table 3
Sample Trailing-Quarter Consumption and Suggested Reorder

Depot	Resource type	Avg. quarterly consumption	Current stock	Suggested reorder
Depot A	Family food packs	320	150	330
Depot B	Medical kits	85	200	0

System Development Methodology

This study adopts the Scrum framework, an iterative and incremental approach that organizes work into short, time-boxed sprints, enabling continuous testing and refinement of the allocation engine, offline-first PWA layer, and inventory dashboard. Each sprint follows the standard Scrum cycle of Sprint Planning, sprint execution with Daily Scrum check-ins, a Sprint Review, and a Sprint Retrospective, drawing from a prioritized Product Backlog.

Product Backlog

Requirements were gathered through stakeholder consultation with CDRRMO San Pablo City's Logistics section, which confirmed the office's current manual processes, its top-priority automation need (inventory tracking), and specific functional requirements such as offline continuity and the

consumable/borrowed-item accountability distinction. These requirements were organized into the Product Backlog as prioritized user stories, with inventory tracking given top priority per the Logistics section's own ranking, then refined and re-prioritized throughout the project via ongoing backlog grooming.

Sprint Planning

The system's architecture, database schema (using an app_-prefixed table convention for all domain models), API contracts between the Laravel backend and the Python solver-service, and the offline-sync data flow were planned during early sprints,, establishing the blueprint followed during implementation.

Sprint Execution

Implementation is carried out across three coordinated codebases: the Laravel 13 + Inertia + React/TypeScript web application (auth, domain logic, real-time broadcasting), the FastAPI solver-service (PuLP replenishment solver), and supporting Docker Compose infrastructure (PostgreSQL, Reverb). Development proceeds iteratively, module by module, following the planned architecture. Brief Daily Scrum check-ins coordinate progress and surface blockers throughout each sprint.

Sprint Review

Each sprint concludes with a Sprint Review, where completed increments are demonstrated and validated. Testing covers unit-level verification of the

solver and backend logic, integration testing between the Laravel backend and the solver-service, and user acceptance testing with CDRRMO San Pablo City personnel, detailed further in the Software Testing section below.

Sprint Retrospective and Deployment

Following each Sprint Review, a Sprint Retrospective identifies process improvements to carry into the next Sprint Planning session. The system is deployed via Docker Compose, with the backend, queue worker, scheduler, Reverb broadcasting server, solver-service, and PostgreSQL database running as separate containerized services, enabling CDRRMO San Pablo City to run the full stack in its own environment.

Software Tools Used

Laravel 13. PHP web application framework used for the domain logic, authentication (Sanctum), and Inertia-based server-driven React frontend.

Inertia.js + React (TypeScript). Used to build the coordinator-facing dashboard, depot management pages, and the resource-request/allocation-approval interface without building a separate REST-consuming SPA.

PostgreSQL. Relational database storing all app_-prefixed domain tables (depots, resources, requests, allocations).

FastAPI. Python web framework used to expose the solver-service's optimization endpoints.

PuLP. Python linear-programming modeling library used to formulate and solve budget-to-resource-type allocation problem via the CBC solver.

Laravel Reverb. Self-hosted WebSocket server used to broadcast real-time stock alerts and dashboard updates to connected coordinators.

IndexedDB. Browser-based local storage API used by the offline-first PWA to queue field data entries during telecommunication blackouts.

Docker & Docker Compose. Used to containerize and orchestrate all services (backend, queue worker, scheduler, Reverb, solver-service,and PostgreSQL) for consistent development and deployment.

Inventory Burn-Rate & Reorder Reporting. The same reporting pipeline used for other generated reports (dompdf for PDF, CSV export) is extended to cover a depot/resource-type inventory burn-rate and reorder-suggestion report, drawing on the quarterly stock snapshots described above.

Hardware Requirements

Development Desktop/Laptop. Used to perform development, containerized service orchestration, and local testing of the full stack. It must have sufficient specifications to run Docker Desktop, a code editor, and multiple concurrent containers (backend, database, solver-service)..

Table 4
Development Machine Minimum Specification

Processor	Intel Core i5 (8th Gen) or AMD Ryzen 5 equivalent
Memory	8 Gigabytes RAM (16 GB recommended for running the full Docker Compose stack)
Storage	15 Gigabytes free disk space (Docker images and PostgreSQL database storage)
System Type	64-bit Operating System

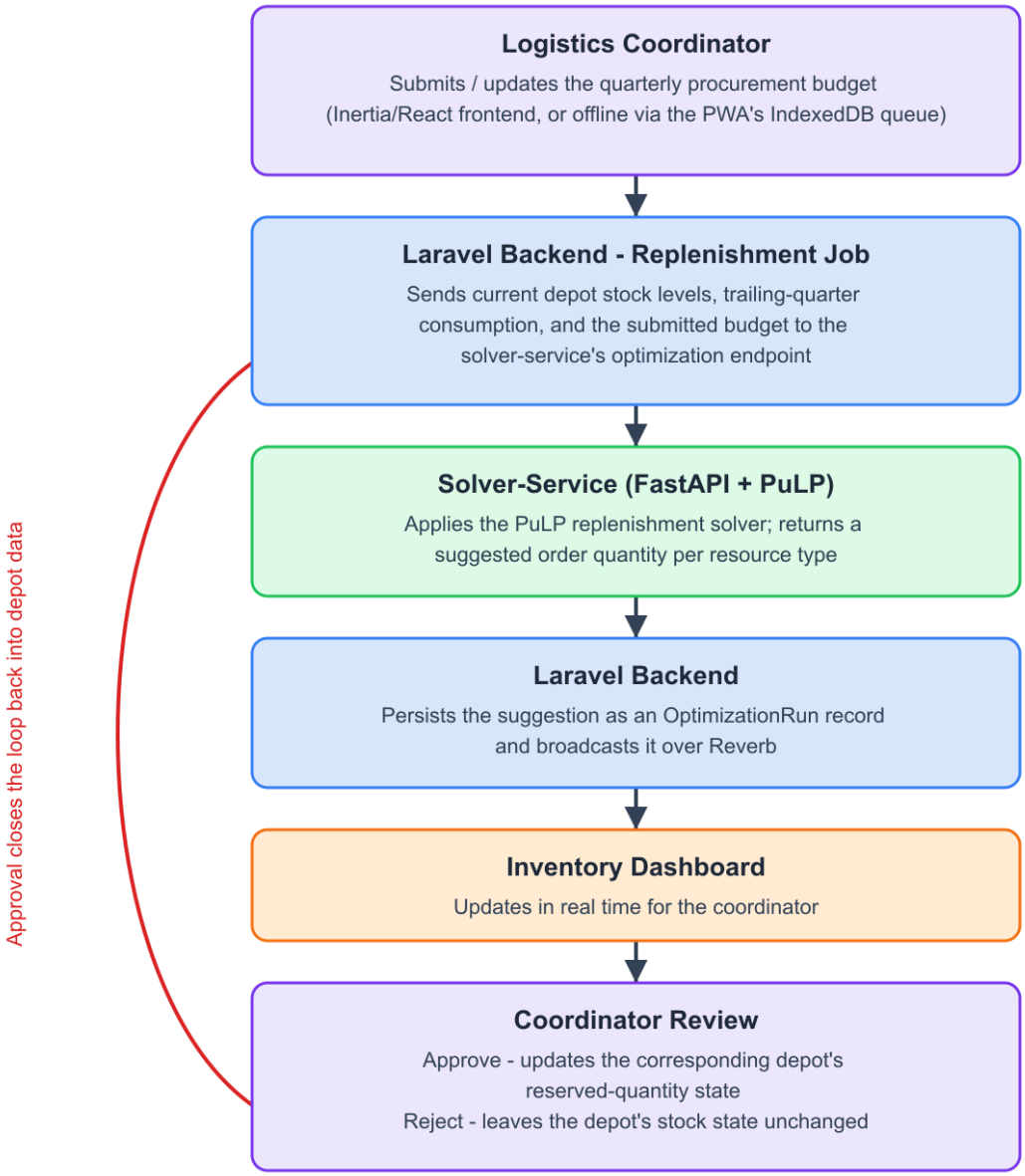
System Architecture

DisasterResource+'s architecture is composed of four main components: the Laravel web application (coordinator-facing domain logic and UI), the solver-service microservice (budget-constrained inventory replenishment optimization), the PostgreSQL database (persistent domain data), and supporting infrastructure services (Reverb for real-time broadcasting, Docker Compose for service orchestration). The architecture follows a client-server model extended with an internal service-to-service call: the Laravel backend acts as the client to the solver-service, which itself has no direct database access and returns only computed suggestions.

Data flows as follows: a Logistics coordinator submits or updates the next quarter's procurement budget through the Inertia/React frontend (or offline through the PWA's IndexedDB queue, synchronized once connectivity returns). The backend's queued replenishment job sends current depot stock levels, trailing-quarter consumption, and the submitted budget to the solver-service's optimization endpoint. The solver-service applies the PuLP replenishment solver, returning a suggested order quantity per resource type, which the backend

persists as an OptimizationRun record and broadcasts over Reverb to update the inventory dashboard. A coordinator then reviews and approves or rejects the suggestion; approval updates the corresponding depot's reserved-quantity state, closing the loop back into the system's data. (Figure 3 illustrates this architecture and data flow.)

Figure 3
System Architecture of DisasterResource+



Software Testing

Testing strategies for DisasterResource+ include unit testing for individual backend and solver-service functions, integration testing for the Laravel-to-solver-service communication, performance testing for the allocation

engine's responsiveness under increasing problem sizes, and user acceptance testing with CDRRMO San Pablo City personnel to evaluate the system against the ISO/IEC 25010 standard, per Research Objective 4.

Unit Testing Procedure

Backend logic (e.g., allocation-approval state transitions) will be tested using PHPUnit, while the solver-service's PuLP replenishment solver will be tested using pytest, following the existing solver-service test suite convention (e.g., tests verifying the PuLP solver fully allocates the submitted budget within depot and cost constraints). Test cases will include both typical inputs (adequate budget for all resource types) and edge cases (zero remaining budget, a resource type with no consumption history).

Integration Testing Procedure

Integration testing validates the interaction between the Laravel backend and the solver-service, simulating the full optimization flow: a budget submission triggering the replenishment job, whose HTTP call to the solver-service, and the resulting OptimizationRun being persisted and broadcast. This will be tested using Laravel's built-in HTTP testing tools alongside manual API testing (e.g., via Postman) against the solver-service's endpoints.

Performance Testing Procedure

The solver-service's response time will be measured as the number of depots, and resource types increases, to characterize how allocation-solving time

scales with problem size. The system's real-time dashboard update latency (from OptimizationRun completion to Reverb broadcast delivery) will also be measured to confirm it remains suitable for timely coordinator review.

User Acceptance Testing Procedure

CDRRMO San Pablo City personnel will evaluate the system's usability and functionality by performing representative tasks — logging a depot update, reviewing a suggested allocation, approving or rejecting it, and viewing the inventory dashboard — while providing feedback aligned to the ISO/IEC 25010 characteristics of functional suitability, reliability, and usability.

Sampling Design. A purposive, non-probability sampling method will be used to select CDRRMO San Pablo City personnel across its confirmed functional roles (Incident Management, ERT, Logistics, Operations, Research and Planning, and Admin & Training) who are directly involved in the office's emergency operations, consistent with the study's CDRRMO-only respondent scope. This ensures respondents have the operational context needed to meaningfully evaluate the system. Given the size of CDRRMO San Pablo City's staff across these six functional roles, the study estimates a target of 15 to 20 respondents, with each functional role contributing at least two to three personnel to ensure balanced representation across the office's operational perspectives.

Data Collection Instrument. The primary data collection tool will be a structured survey based on the ISO/IEC 25010 standard, covering functional suitability, reliability, and usability. The instrument will use Likert-scale items to gauge agreement with statements about the system's completeness,

correctness, fault tolerance, and ease of use, supplemented with open-ended items to capture qualitative feedback. Prior to deployment, the instrument will be reviewed for face and content validity and pilot tested with a small group of CDRRMO personnel to refine item clarity.

Validation of the Questionnaire. The questionnaire's face and content validity will be assessed by having it reviewed against the ISO/IEC 25010:2023 standard's characteristic definitions, ensuring each survey item maps clearly to functional suitability, reliability, or usability. Feedback from this review, along with the pilot test, will be used to refine wording and remove ambiguous items before full deployment.

Statistical Treatment. Collected responses will be analyzed using descriptive statistics (mean scores, frequency distributions, and standard deviations) to summarize perceived functional suitability, reliability, and usability. Where relevant, responses will be compared across the different CDRRMO functional roles to check whether perceptions of the system differ meaningfully by role, using appropriate non-parametric or parametric tests depending on the data's distribution.

References

Albano, H. B. (2025). Empowering local government units with open-source tools: Building a dynamic web-based information system. Isabela State University Linker: Journal of Engineering, Computing, and Technology, 1(1), 1–14.

Cuenca-Enrique, C., et al. (2024). Progressive Web Apps: An optimal solution for rural communities in developing countries? In Information Systems and Technologies: WorldCIST 2023 (Lecture Notes in Networks and Systems, Vol. 801). Springer.

Dimez, O., Camci, A., & Acarman, T. (2026). Response improvement during disaster management scenarios: Evaluation of algorithms to assign technical teams and to reactivate cellular communication. Symmetry, 18(5), Article 781. https://doi.org/10.3390/sym18050781

Han, Y., & Liu, G. (2025). The role of GIS in disaster response: Improving efficiency and overcoming challenges. Theoretical and Natural Science.

Heng, S., Phutthanawong, K., & Boonmee, C. (2025). Optimizing localized humanitarian logistics: A stochastic programming approach for facility location, inventory and evacuation strategies. Journal of Humanitarian Logistics and Supply Chain Management. https://doi.org/10.1108/JHLSCM-02-2025-0020

Huber, S., Demetz, L., & Felderer, M. (2022). A comparative study on the energy consumption of Progressive Web Apps. Information Systems, 108, Article 102017.

Husein, S., Nasution, Z. F., Rezeki, E. S., & Ahkrizal, A. (2026). Application of linear programming based transportation models to optimize natural disaster relief distribution. ZERO: Jurnal Sains, Matematika dan Terapan, 10(1).

Ilabaca, A., Paredes-Belmar, G., & Álvarez, P. P. (2022). Optimization of humanitarian aid distribution in case of an earthquake and tsunami in the city of Iquique, Chile. Sustainability, 14(2), Article 819.

International Organization for Standardization. (2023). ISO/IEC 25010:2023 — Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — Product quality model.

Lata, K., Sood, A., Kaur, K., Benipal, A. K., & Pateriya, B. (2024). Designing interactive Web-GIS dashboards for data-driven smart city emergency management. Journal of Logistics, Informatics and Service Science, 11(2), 357–373.

Macose, V. G. (2024). Building a resilient community with an ICT-powered Local Government Unit (LGU): Community-based disaster solution. Jurnal Pendidikan Geografi: Kajian, Teori, dan Praktek dalam Bidang Pendidikan dan Ilmu Geografi, 27(2), 219–227.

Melvin, C., & Nguyen, N. R. (2025). AWARE: Evaluating PriorityFresh caching for offline emergency warning systems. arXiv preprint arXiv:2511.05022.

Osio, R. H. S., Ilustrisimo, D. L., & Daruca, R. H. (2026). A web-based customer satisfaction survey system for local government unit: Design, implementation, and ISO/IEC 25010 quality evaluation. International Journal of Education, Research, and Innovation Perspectives, 2(3), 356–363.

Park, H. S., Kwon, S. A., & Azam, M. (2024). A study on GIS-based spatial analysis of emergency response for disaster management: Focusing on Seoul. Heliyon, 10(7).

Philippine Council for Health Research and Development. (n.d.). E-LIGTAS: A disaster response application. Department of Science and Technology.

Pratama, A. A., & Mutiara, A. B. (2021). Software quality analysis for Halodoc application using ISO 25010:2011. International Journal of Advanced Computer Science and Applications, 12(8).

Rojas Enriquez, H., Renteria, R., Martinez Duran, V., Tomaylla Gutiérrez, Y., Ibarra Cabrera, M. J., & Aminuddin, A. (2025). Mapping the evolution and future directions of ISO/IEC 25010: A bibliometric and thematic analysis. Engineering, Technology & Applied Science Research, 15(5), 27530–27541.

Wang, G. (2024). Disaster relief supply chain network planning under uncertainty. Annals of Operations Research, 338, 1127–1156.

Yajie, W., Jianchun, F., Shengnan, W., & Yunpeng, Y. (2023). A heuristic algorithm optimizing emergency resource storage and location in the South China Sea. Ocean Engineering, 268.

Yang, Z., Li, J., Hyyppä, J., Gong, J., Liu, J., & Yang, B. (2023). A comprehensive and up-to-date web-based interactive 3D emergency response and visualization system using Cesium Digital Earth: Taking landslide disaster as an example. Big Earth Data, 7(4).